Actividad 3 – Programación III

Mejora de programas en Prolog mediante el uso de listas

Universidad Tecnológica de Pereira
Facultad de Ingenierías
Ingeniería de Sistemas y Computación

Introducción

En esta actividad se mejoran dos programas desarrollados en **SWI-Prolog** incorporando el uso de **listas**: el árbol genealógico de la familia Simpson y el problema de viajes entre ciudades con costos. Para cada programa se presenta el código fuente completo (hechos y reglas), las consultas de validación y los resultados obtenidos al ejecutarlas.

1 Programa 1: Árbol genealógico (Familia Simpson)

Descripción de la mejora

El programa original resolvía las relaciones de varias generaciones (hermano, tío, primo, abuelo) únicamente con hechos y reglas, devolviendo una respuesta por solución y con resultados repetidos. La versión mejorada agrega:

- **Hechos con listas:** `familia/2` agrupa a los miembros de cada apellido en una lista.
- **Reglas que devuelven listas:** `hijos/2`, `progenitores/2`, `hermanos/2`, `tios/2`, `primos/2` y `abuelos/2`, construidas con `findall/3` y `sort/2` (que además elimina duplicados).
- **Reglas recursivas sobre listas:** `descendientes/2` y `ancestros/2`, que recorren varias generaciones con `append/3`.
- **Reglas que consultan listas:** `numero_hijos/2` (con `length/2`), `apellido/2`, `mismo_apellido/2` y `son_hermanos/2` (con `member/2`).

Código fuente: `simpsons.pl`

```
%-------------Hechos------------

%padres primera generacion
padre(abraham, herbert).
padre(abraham, homero).
padre(clancy, marge).
padre(clancy, patty).
padre(clancy, selma).

%padres segunda generacion
padre(homero, bart).
```

```
padre(homero, lisa).
padre(homero, maggie).

%madres primera generacion
madre(mona, herbert).
madre(mona, homero).
madre(jacqueline, marge).
madre(jacqueline, patty).
madre(jacqueline, selma).

%madres segunda generacion
madre(marge, bart).
madre(marge, lisa).
madre(marge, maggie).
madre(selma, ling).

%--------------------------------


%------------Reglas-----------
hermano(Hermanox, Hermanoy) :-
        (   padre(Padre, Hermanox), padre(Padre, Hermanoy), Hermanox \=
           Hermanoy ) ; %hermano por parte de papa
    (   madre(Madre, Hermanox), madre(Madre, Hermanoy), Hermanox \=
       Hermanoy ). %hermano por parte de mama

tio(Tio, Yo) :-
    (   padre(Padre, Yo), hermano(Padre, Tio), Tio \= Padre  ); %tio por
       parte de papa
    (   madre(Madre, Yo), hermano(Madre, Tio), Tio \= Madre  ). %tio por
       parte de mama

primo(Yo, Primo) :-
    (   padre(Tio, Primo), tio(Tio, Yo)  ); %primo por parte de papa
    (   madre(Tio, Primo), tio(Tio, Yo)  ). %primo por parte de mama

abuelo(Abuelo, Yo) :-
    (   padre(Padre, Yo), (   padre(Abuelo, Padre); madre(Abuelo, Padre)),
        Abuelo \= Padre); %tio por parte de papa
    (   madre(Madre, Yo), (   madre(Abuelo, Madre); padre(Abuelo, Madre)),
        Abuelo \= Madre). %tio por parte de mama


%============================================================
%   MEJORAS CON LISTAS
%============================================================

%------------Hechos con listas-----------
% Cada familia (apellido) agrupa a sus miembros en una lista
familia(simpson, [abraham, mona, homero, marge, bart, lisa, maggie]).
familia(bouvier, [clancy, jacqueline, patty, selma, ling]).
familia(powell,  [herbert]).

%------------Reglas auxiliares-----------
```

```
% progenitor/2: unifica padre y madre en una sola relacion
progenitor(P, H) :- padre(P, H).
progenitor(P, H) :- madre(P, H).

% apellido/2: usa member/2 para buscar a la persona en la lista de su
    familia
apellido(Persona, Apellido) :-
    familia(Apellido, Miembros),
    member(Persona, Miembros).

%------------Reglas que devuelven listas-----------
% findall/3 junta todas las respuestas y sort/2 elimina duplicados
% (las reglas originales repiten resultados cuando hay hermanos
% por padre y por madre a la vez).
hijos(Persona, Lista) :-
    findall(H, progenitor(Persona, H), L0),
    sort(L0, Lista).

progenitores(Persona, Lista) :-
    findall(P, progenitor(P, Persona), L0),
    sort(L0, Lista).

hermanos(Persona, Lista) :-
    findall(H, hermano(Persona, H), L0),
    sort(L0, Lista).

tios(Persona, Lista) :-
    findall(T, tio(T, Persona), L0),
    sort(L0, Lista).

primos(Persona, Lista) :-
    findall(P, primo(Persona, P), L0),
    sort(L0, Lista).

abuelos(Persona, Lista) :-
    findall(A, abuelo(A, Persona), L0),
    sort(L0, Lista).

%------------Reglas recursivas sobre listas-----------
% descendientes/2: hijos, nietos, bisnietos, ...
descendientes(Persona, Lista) :-
    hijos(Persona, Hijos),
    descendientes_de_todos(Hijos, Nietos),
    append(Hijos, Nietos, L0),
    sort(L0, Lista).

descendientes_de_todos([], []).
descendientes_de_todos([H|T], Lista) :-
    descendientes(H, DH),
    descendientes_de_todos(T, DT),
    append(DH, DT, Lista).

% ancestros/2: padres, abuelos, bisabuelos, ...
ancestros(Persona, Lista) :-
```

```
    progenitores(Persona, Padres),
    ancestros_de_todos(Padres, Abuelos),
    append(Padres, Abuelos, L0),
    sort(L0, Lista).

ancestros_de_todos([], []).
ancestros_de_todos([P|T], Lista) :-
    ancestros(P, AP),
    ancestros_de_todos(T, AT),
    append(AP, AT, Lista).

%------------Reglas que consultan listas-----------
% numero_hijos/2: usa length/2
numero_hijos(Persona, N) :-
    hijos(Persona, Lista),
    length(Lista, N).

% mismo_apellido/2: ambos pertenecen a la misma lista de familia
mismo_apellido(P1, P2) :-
    apellido(P1, A),
    apellido(P2, A),
    P1 \= P2.

% son_hermanos/2: verifica pertenencia con member/2
son_hermanos(P1, P2) :-
    hermanos(P1, Lista),
    member(P2, Lista).
```

Consultas de validación

```
?- hijos(homero, L).
L = [bart, lisa, maggie].

?- progenitores(bart, L).
L = [homero, marge].

?- hermanos(bart, L).
L = [lisa, maggie].

?- tios(bart, L).
L = [herbert, patty, selma].

?- primos(bart, L).
L = [ling].

?- abuelos(bart, L).
L = [abraham, clancy, jacqueline, mona].

?- descendientes(abraham, L).
L = [bart, herbert, homero, lisa, maggie].

?- ancestros(bart, L).
L = [abraham, clancy, homero, jacqueline, marge, mona].

```

```
?- numero_hijos(homero, N).
N = 3.

?- apellido(herbert, A).
A = powell.

?- mismo_apellido(bart, abraham).
true.

?- mismo_apellido(bart, patty).
false.

?- son_hermanos(lisa, maggie).
true.

?- son_hermanos(bart, ling).
false.
```

Análisis de resultados

Las reglas que devuelven listas entregan todas las respuestas en una sola consulta y sin duplicados (por ejemplo, `hermanos(bart, L)` da `[lisa, maggie]`, mientras que la regla original `hermano/2` repetía cada resultado). Las reglas recursivas `descendientes/2` y `ancestros/2` recorren todas las generaciones disponibles: los ancestros de Bart incluyen padres y abuelos. Las consultas `mismo_apellido/2` y `son_hermanos/2` comprueban correctamente casos verdaderos y falsos.

2 Programa 2: Viajes entre ciudades con listas

Descripción del programa

Se tiene un conjunto de ciudades conectadas por rutas directas con un costo asociado, representadas con hechos `conexion(Origen, Destino, Costo)`. La regla `viajar/4` determina si es posible viajar de un origen a un destino, calculando el **costo total** y el **camino** recorrido como una lista de ciudades. Para evitar ciclos infinitos, la regla recursiva usa una lista de ciudades visitadas y `member/2` para no repetir ciudades; el camino se acumula en orden inverso y se invierte al final con `reverse/2`. Se incluye también `viajar/3`, que devuelve solo el costo.

Código fuente: `ciudades_con_listas.pl`

```
%----------Hechos-------------

%conexiones directas
conexion(vancouver,edmonton,16).
conexion(vancouver,calgary,13).
conexion(edmonton,saskatoon,12).
conexion(calgary,regina, 14).
conexion(calgary,edmonton, 4).
conexion(saskatoon,calgary, 9).
conexion(saskatoon,winnipeg, 20).
conexion(regina,saskatoon, 7).
conexion(regina,winnipeg, 4).
```

```
13
14
15 %------------Reglas-------------
16
17 % Camino es la lista de ciudades recorridas, de Origen a Destino.
18 viajar(Origen, Destino, Costo, Camino) :-
19     viajar_aux(Origen, Destino, [Origen], Costo, CaminoInverso),
20     reverse(CaminoInverso, Camino).
21
22 viajar(Origen, Destino, Costo) :-
23     viajar(Origen, Destino, Costo, _).
24
25 viajar_aux(Actual, Destino, Visitadas, Costo, [Destino|Visitadas]) :-   %
       Caso base
26     conexion(Actual, Destino, Costo),
27     \+ member(Destino, Visitadas).
28
29 viajar_aux(Actual, Destino, Visitadas, Costo, Camino) :-                %
       Caso recursivo
30     conexion(Actual, Intermediario, Costo1),
31     \+ member(Intermediario, Visitadas),                                 %
           no repetir ciudades
32     viajar_aux(Intermediario, Destino, [Intermediario|Visitadas], Costo2,
           Camino),
33     Costo is Costo1 + Costo2.
```

Consultas de validación

```
1  ?- viajar(vancouver, winnipeg, Costo, Camino).
2  Costo = 48, Camino = [vancouver, edmonton, saskatoon, winnipeg] ;
3  Costo = 55, Camino = [vancouver, edmonton, saskatoon, calgary, regina,
       winnipeg] ;
4  Costo = 31, Camino = [vancouver, calgary, regina, winnipeg] ;
5  Costo = 54, Camino = [vancouver, calgary, regina, saskatoon, winnipeg] ;
6  Costo = 49, Camino = [vancouver, calgary, edmonton, saskatoon, winnipeg].
7
8  ?- viajar(vancouver, regina, Costo, Camino).
9  Costo = 51, Camino = [vancouver, edmonton, saskatoon, calgary, regina] ;
10 Costo = 27, Camino = [vancouver, calgary, regina].
11
12 ?- viajar(vancouver, saskatoon, Costo).
13 Costo = 28 ;
14 Costo = 34 ;
15 Costo = 29.
16
17 ?- viajar(calgary, winnipeg, Costo, Camino).
18 Costo = 18, Camino = [calgary, regina, winnipeg] ;
19 Costo = 41, Camino = [calgary, regina, saskatoon, winnipeg] ;
20 Costo = 36, Camino = [calgary, edmonton, saskatoon, winnipeg].
21
22 ?- viajar(regina, calgary, Costo, Camino).
23 Costo = 16, Camino = [regina, saskatoon, calgary].
24
25 ?- viajar(winnipeg, vancouver, Costo, Camino).
```

```
26 false.
```

Análisis de resultados

Entre Vancouver y Winnipeg existen cinco rutas distintas sin repetir ciudades; la de menor costo es `[vancouver, calgary, regina, winnipeg]` con costo 31. La consulta `viajar(winnipeg, vancouver, ...)` falla porque las conexiones son dirigidas y no existe ninguna ruta de regreso desde Winnipeg. La lista de ciudades visitadas evita que el programa entre en ciclos, a pesar de que el grafo contiene caminos circulares (por ejemplo, entre Saskatoon, Calgary y Regina).

Repositorio GitHub

El código fuente completo de la actividad, junto con este informe, se encuentra disponible en el siguiente repositorio:

https://github.com/Keleseth98/Actividad-3-programacion-3